**DAA**

**Algorithm Analysis Fundamentals**

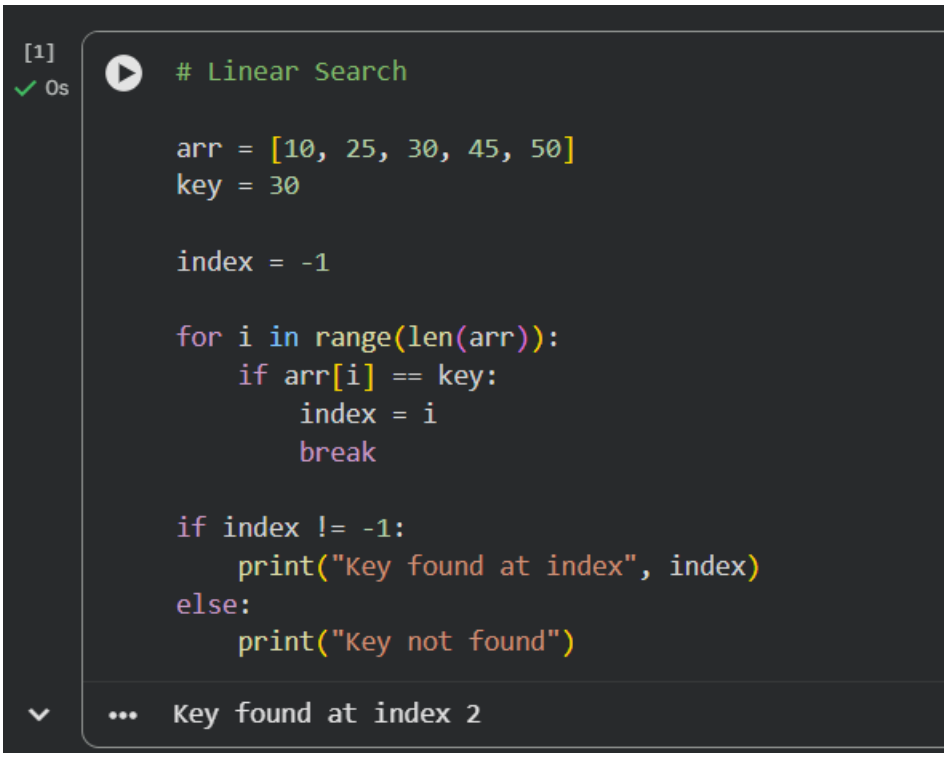
Time complexity analysis of iterative and recursive algorithms; verify Big-O, Ω, Θ for sample programs

Dr. P. SRIRAMYA

Name: Sairam D  Reg. No.: 192424111

## Experiment 1: Linear Search (Iterative)

- **Objective:** To analyze time complexity of linear search.
- **Problem Statement:** Write a Python program to search for a key in an array using linear search.
- **Sample Input:** `[10, 25, 30, 45, 50]`, Key = `30`
- **Sample Output:** `Key found at index 2`
- **Complexity:** Ω: O(1), O: O(n), Θ: Θ(n)
- **Explanation:** Demonstrates sequential scanning and inefficiency for large inputs.



## Experiment 2: Binary Search (Recursive)

- **Objective:** To compare recursive binary search complexity.
- **Problem Statement:** Implement recursive binary search in Python.
- **Sample Input:** `[5, 10, 15, 20, 25]`, Key = `20`
- **Sample Output:** `Key found at index 3`
- **Complexity:** Ω: O(1), O: O(log n), Θ: Θ(log n)
- **Explanation:** Shows divide-and-conquer and logarithmic efficiency.

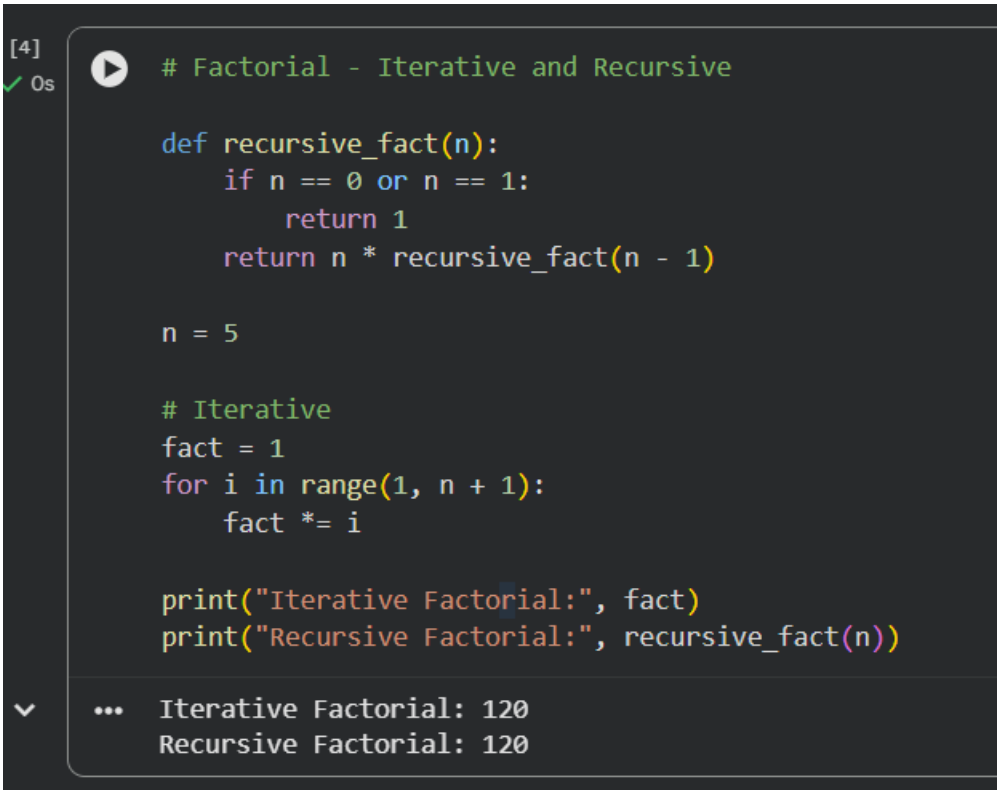
[3]
✓ 0s

```python
# Recursive Binary Search

def binary_search(arr, low, high, key):
    if low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] > key:
            return binary_search(arr, low, mid - 1, key)
        else:
            return binary_search(arr, mid + 1, high, key)
    return -1

arr = [5, 10, 15, 20, 25]
key = 20

result = binary_search(arr, 0, len(arr)-1, key)

if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")
```

```
Key found at index 3
```

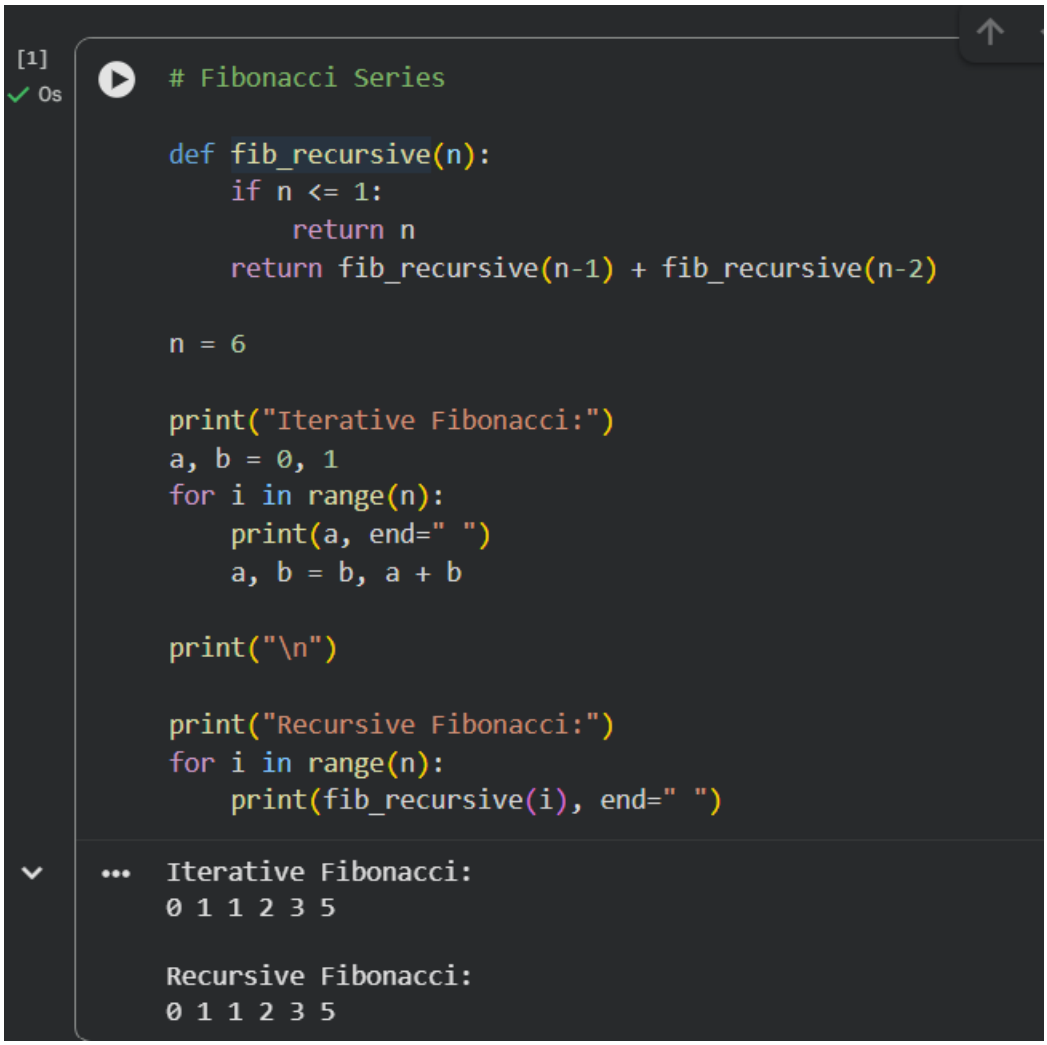
## Experiment 3: Factorial (Iterative vs Recursive)

- **Objective:** To compare iterative and recursive factorial computation.
- **Problem Statement:** Write programs to compute factorial using both methods.
- **Sample Input:** `n = 5`
- **Sample Output:** `120`
- **Complexity:** Both O(n)
- **Explanation:** Highlights recursion depth vs iterative loops.



## Experiment 4: Fibonacci Series

- **Objective:** To analyze recursive vs iterative Fibonacci generation.
- **Problem Statement:** Generate first n Fibonacci numbers.
- **Sample Input:** `n = 6`
- **Sample Output:** `[0, 1, 1, 2, 3, 5]`
- **Complexity:** Iterative O(n), Recursive naïve O(2n), Recursive with memoization O(n)
- **Explanation:** Illustrates exponential recursion vs efficient iteration.



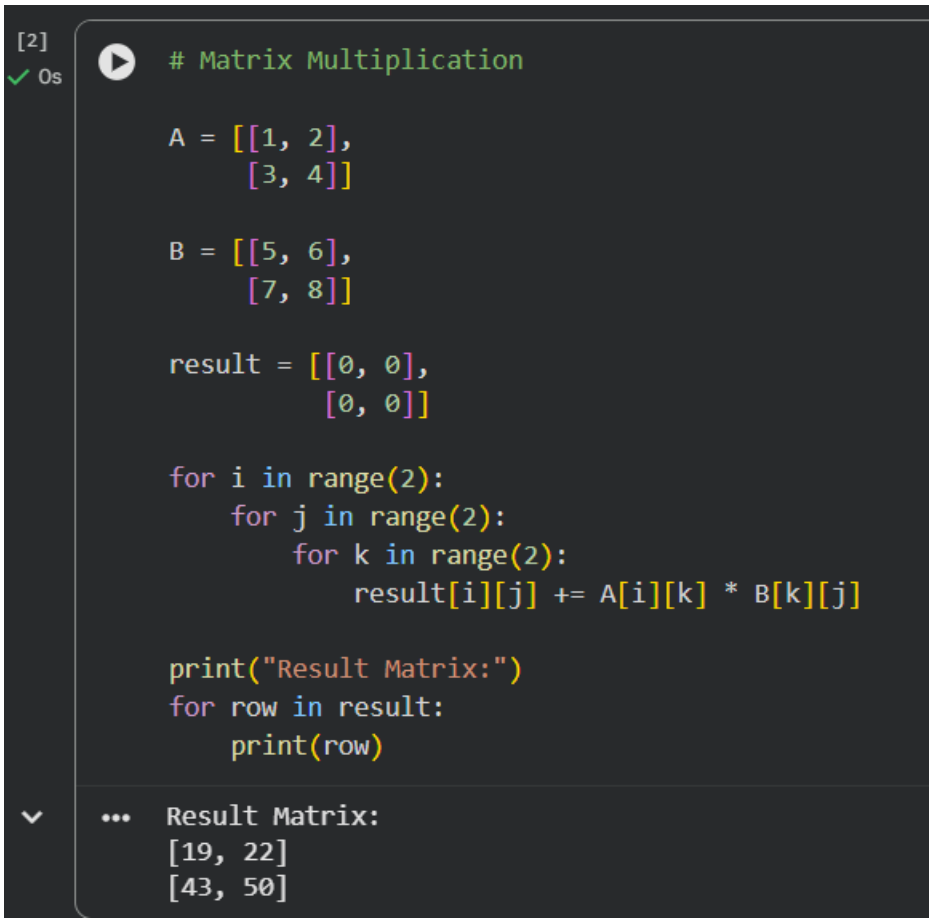
```
[1]
✓ 0s
```

```python
# Fibonacci Series

def fib_recursive(n):
    if n <= 1:
        return n
    return fib_recursive(n-1) + fib_recursive(n-2)

n = 6

print("Iterative Fibonacci:")
a, b = 0, 1
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b

print("\n")

print("Recursive Fibonacci:")
for i in range(n):
    print(fib_recursive(i), end=" ")
```

```
Iterative Fibonacci:
0 1 1 2 3 5

Recursive Fibonacci:
0 1 1 2 3 5
```

## Experiment 5: Matrix Multiplication

- **Objective:** To analyze nested loop complexity.
- **Problem Statement:** Multiply two matrices using loops.
- **Sample Input:** `A=[[1,2],[3,4]], B=[[5,6],[7,8]]`
- **Sample Output:** `[[19,22],[43,50]]`
- **Complexity:** O(n3)
- **Explanation:** Shows cubic growth due to nested loops.



## Experiment 6: Merge Sort

- **Objective:** To analyze divide-and-conquer sorting.
- **Problem Statement:** Implement merge sort.
- **Sample Input:** `[38, 27, 43, 3, 9, 82, 10]`
- **Sample Output:** `[3, 9, 10, 27, 38, 43, 82]`
- **Complexity:** Always O(nlog⁡n)
- **Explanation:** Demonstrates guaranteed efficiency of divide-and-conquer.

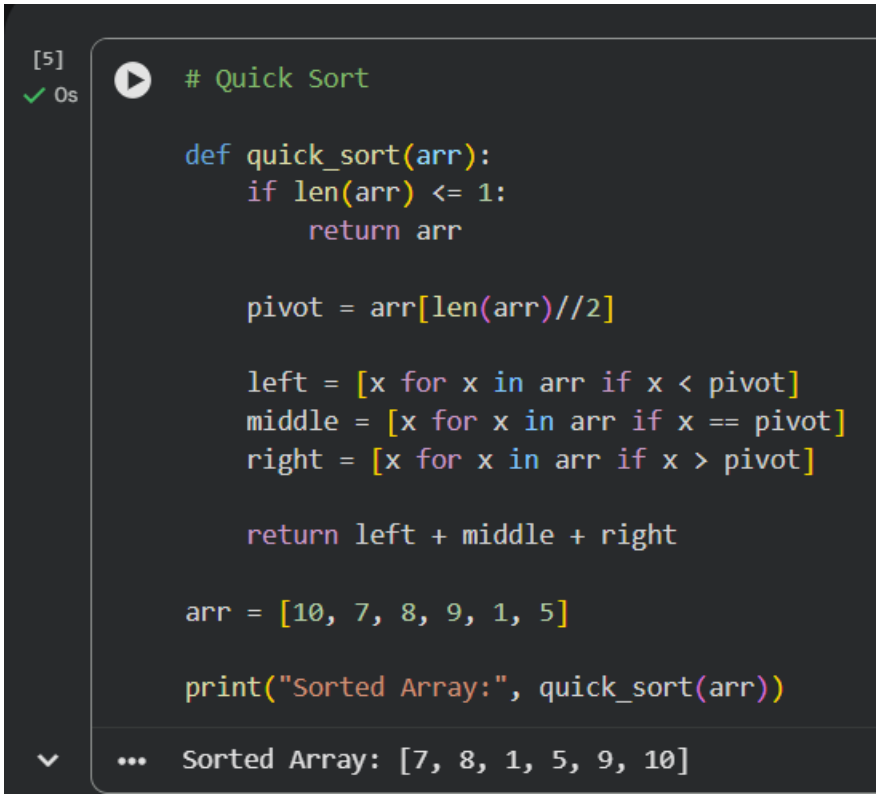
[3]
✓ 0s

```python
# Merge Sort

def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

arr = [38, 27, 43, 3, 9, 82, 10]

merge_sort(arr)

print("Sorted Array:", arr)
```

```
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
```

## Experiment 7: Quick Sort

- **Objective:** To analyze recursive quick sort.
- **Problem Statement:** Implement quick sort.
- **Sample Input:** `[10, 7, 8, 9, 1, 5]`
- **Sample Output:** `[1, 5, 7, 8, 9, 10]`
- **Complexity:** Ω: O(nlogn), O: O(n2), Θ: Θ(nlogn)
- **Explanation:** Shows how pivot choice affects performance.



## Experiment 8: Tower of Hanoi

- **Objective:** To analyze exponential recursion.
- **Problem Statement:** Solve Tower of Hanoi for n disks.
- **Sample Input:** n = 3
- **Sample Output:** Sequence of moves.
- **Complexity:** O(2n)
- **Explanation:** Illustrates exponential growth in recursive calls.

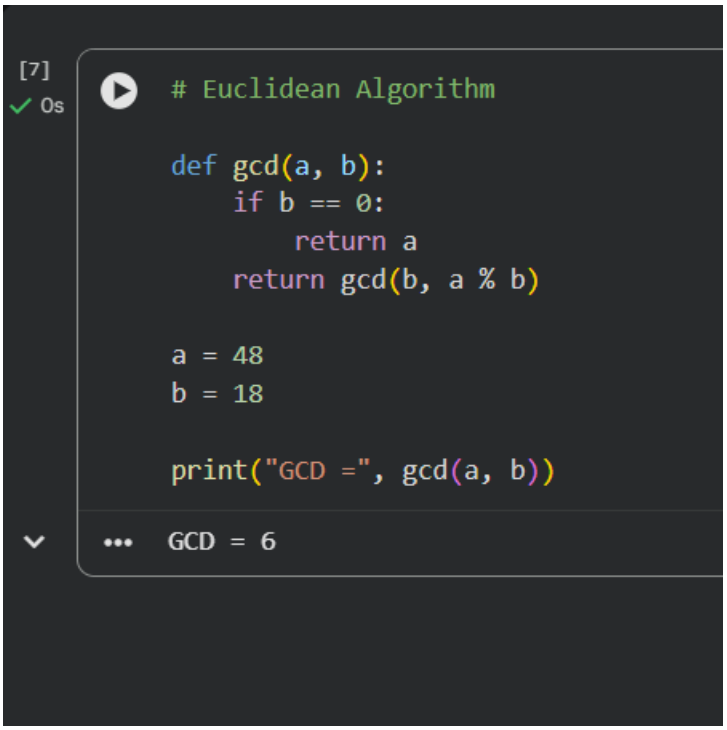
[6]
✓ 0s

```python
# Tower of Hanoi

def tower(n, source, auxiliary, destination):
    if n == 1:
        print("Move disk 1 from", source, "to", destination)
        return

    tower(n-1, source, destination, auxiliary)
    print("Move disk", n, "from", source, "to", destination)
    tower(n-1, auxiliary, source, destination)

n = 3

tower(n, 'A', 'B', 'C')
```

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

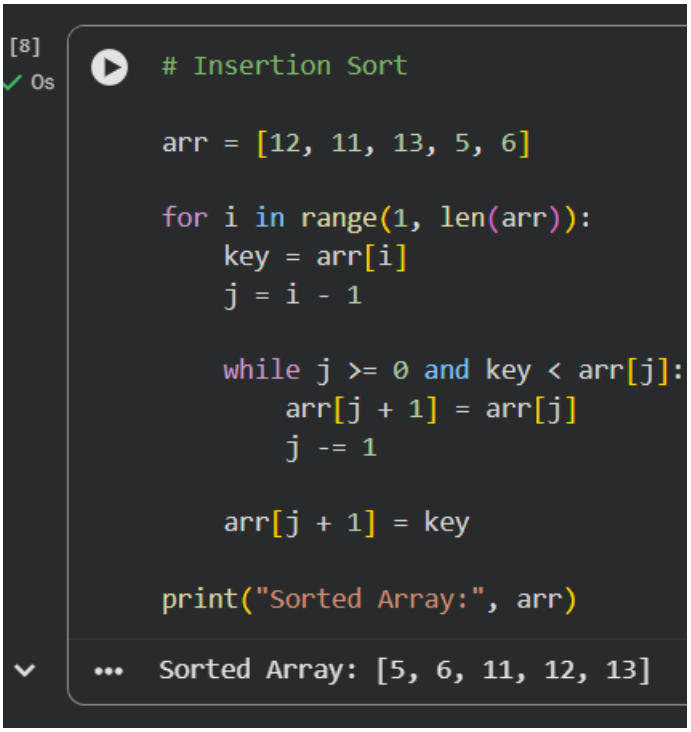
## Experiment 9: GCD (Euclidean Algorithm)

- **Objective:** To analyze recursive GCD.
- **Problem Statement:** Implement Euclidean algorithm.
- **Sample Input:** a=48， b=18
- **Sample Output:** 6
- **Complexity:** O(log□n)
- **Explanation:** Shows efficient recursion with logarithmic complexity.



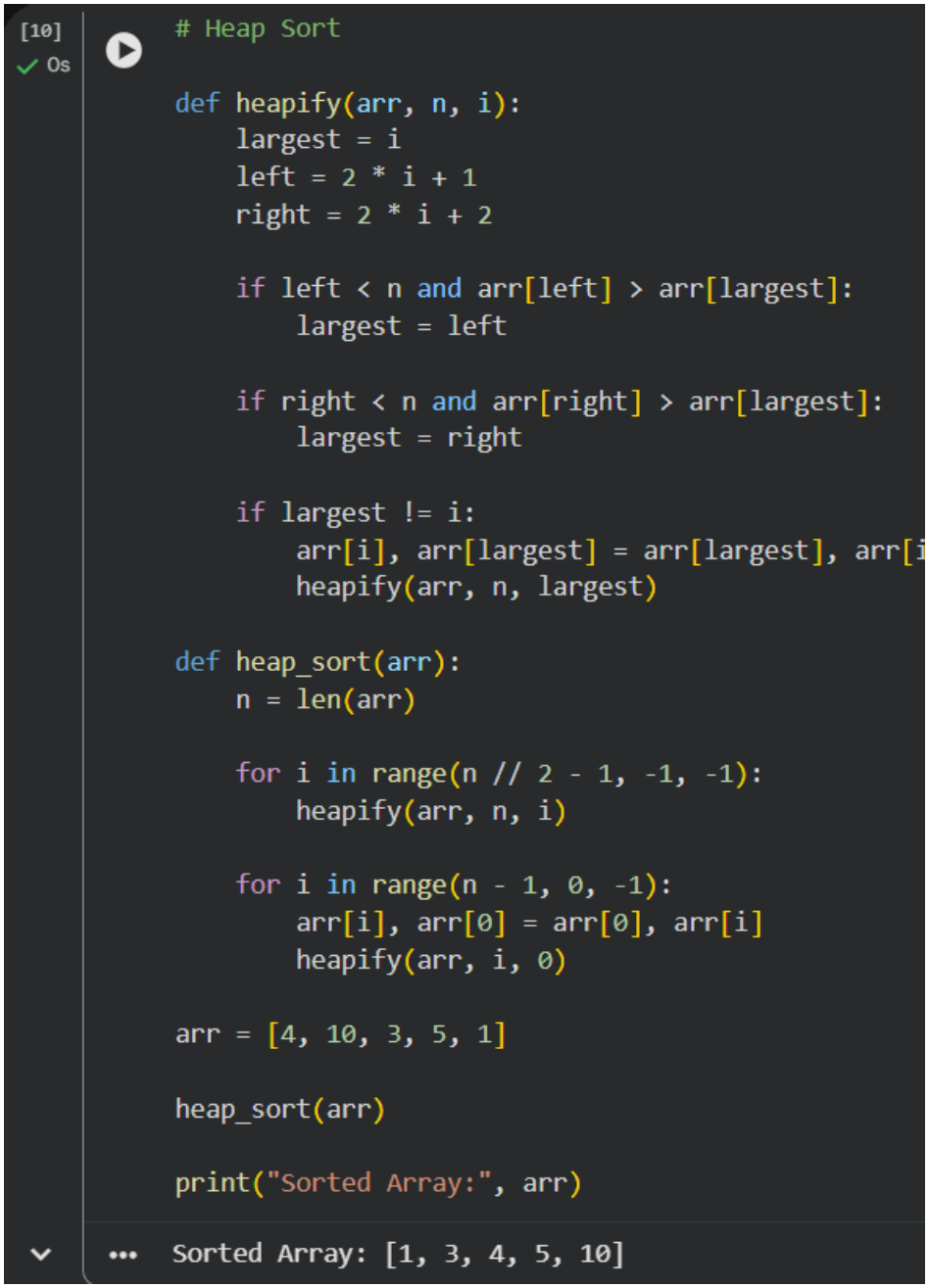
## Experiment 10: Insertion Sort

- **Objective:** To analyze iterative sorting.
- **Problem Statement:** Implement insertion sort.
- **Sample Input:** [12, 11, 13, 5, 6]
- **Sample Output:** [5, 6, 11, 12, 13]
- **Complexity:** Ω: O(n), O: O(n2), Θ: Θ(n2)
- **Explanation:** Demonstrates input sensitivity (sorted vs unsorted).



## Experiment 11: Heap Sort

- **Objective:** To analyze heap-based sorting.
- **Problem Statement:** Implement heap sort.
- **Sample Input:** `[4, 10, 3, 5, 1]`
- **Sample Output:** `[1, 3, 4, 5, 10]`
- **Complexity:** Always O(nlogn)
- **Explanation:** Consistent performance using heap structure.



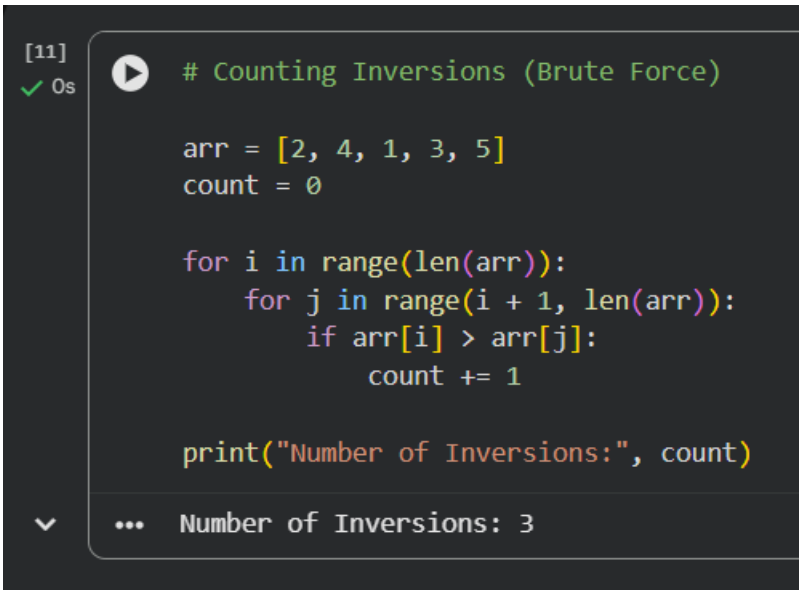
```
[10]
✓ 0s
```

```python
# Heap Sort

def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)

    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)

arr = [4, 10, 3, 5, 1]

heap_sort(arr)

print("Sorted Array:", arr)
```

```
Sorted Array: [1, 3, 4, 5, 10]
```

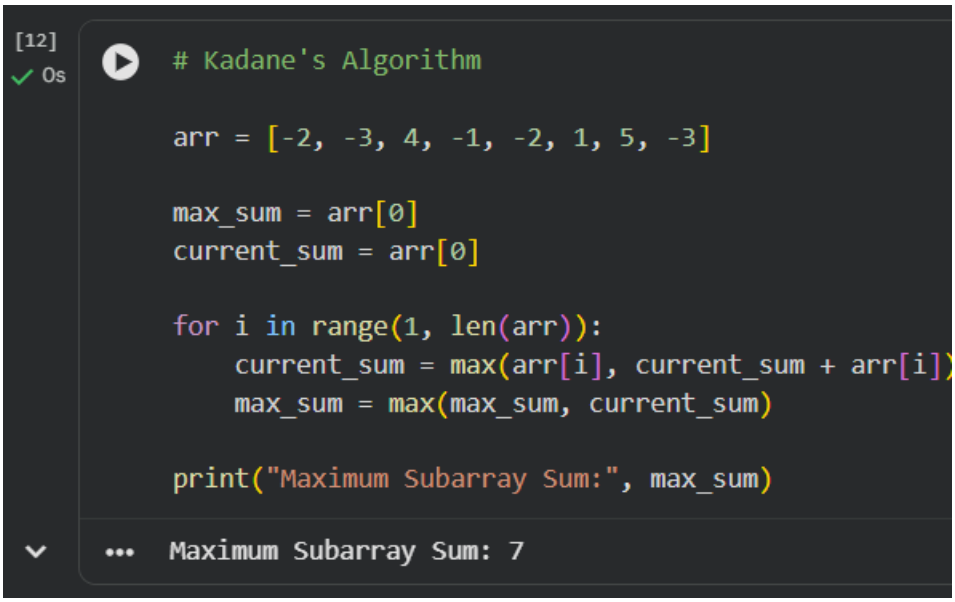
## Experiment 12: Counting Inversions

- **Objective:** To analyze modified merge sort.
- **Problem Statement:** Count inversions in an array.
- **Sample Input:** `[2, 4, 1, 3, 5]`
- **Sample Output:** 3
- **Complexity:** O(nlogn)
- **Explanation:** Shows divide-and-conquer applied beyond sorting.



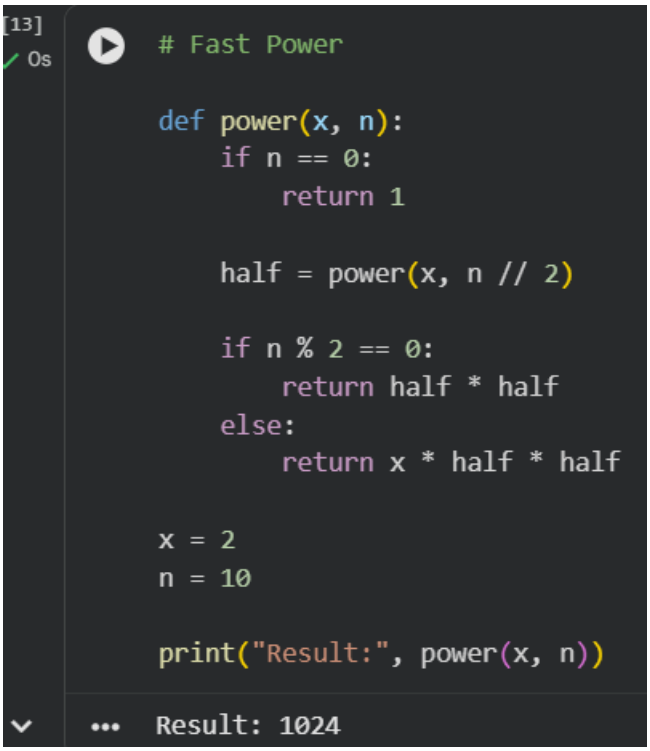
## Experiment 13: Maximum Subarray Sum

- **Objective:** To compare Kadane's vs divide-and-conquer.
- **Problem Statement:** Find maximum subarray sum.
- **Sample Input:** `[-2, -3, 4, -1, -2, 1, 5, -3]`
- **Sample Output:** 7
- **Complexity:** Kadane's O(n), Divide & Conquer O(nlogn)
- **Explanation:** Highlights efficiency differences between approaches.



## Experiment 14: Exponentiation

- **Objective:** To compare iterative vs recursive fast power.
- **Problem Statement:** Compute xn.
- **Sample Input:** `x=2, n=10`
- **Sample Output:** `1024`
- **Complexity:** Iterative O(n), Recursive fast power O(log n)
- **Explanation:** Shows how recursion reduces complexity.



## Experiment 15: Closest Pair of Points

- **Objective:** To analyze brute force vs divide-and-conquer.
- **Problem Statement:** Find closest pair of points in 2D.
- **Sample Input:** [(1,2), (4,5), (7,8), (3,1)]
- **Sample Output:** Closest pair (1,2)–(3,1), Distance = √5
- **Complexity:** Brute force O(n2), Optimized O(nlog n)
- **Explanation:** Demonstrates quadratic complexity and motivates optimization.